

PetFlow 作业报告

1 程序功能介绍

PetFlow 是一个基于任务图的桌面工作流管理程序。与普通待办清单按列表排列任务不同，本项目把任务、习惯、资料、检查点和奖励抽象为图中的节点，把依赖、循环、推荐和触发关系抽象为边。用户可以在画布上直观看到任务之间的先后关系，并通过桌宠和 Agent 辅助完成任务规划、推荐和复盘。

项目使用 Python 3.12 开发，界面基于 Tkinter 和 Canvas 实现，数据通过 JSON 文件进行本地持久化。程序入口为 `src/petflow/main.py`，运行命令为：

```
PYTHONPATH=src python -m petflow.main
```

程序主要功能如下：

1. **任务图编辑。**用户可以创建、编辑、删除和拖拽节点，也可以创建、编辑、删除节点之间的边。节点支持任务、Routine、资源、检查点、奖励等类型；边支持依赖、Routine、推荐、触发等类型。依赖边会进行成环检测，避免出现逻辑错误。
2. **节点信息管理。**每个节点可以记录标题、描述、状态、优先级、预计耗时、实际耗时、标签、检查清单、附件、资源路径、重复规则、下次到期时间等信息。选中节点后，右侧编辑面板可直接修改这些字段。
3. **七日议程。**左侧面板展示未来 7 天的任务和 Routine 到期情况，帮助用户把图结构中的任务转化为近期计划。
4. **本地推荐。**推荐引擎会根据依赖是否完成、节点状态、优先级、Routine 是否到期、当前焦点任务等因素，推荐下一步最适合执行的节点。
5. **桌宠交互。**桌宠不是单纯装饰，而是和任务图状态联动。当用户开始任务、完成任务或请求推荐时，桌宠会移动到对应节点附近，并显示提示气泡。
6. **Agent 辅助规划。**程序支持 OpenAI 兼容接口，也支持 mock 模式。用户可以通过自然语言生成任务图，或让 Agent 拆分已有节点。Agent 输出不会直接修改图，而是先转成 proposal，经字段校验和预览后，再通过图服务应用到任务图。

7. 系统增强能力。程序提供剪贴板捕获功能，可将剪贴板中的 URL 或文本转成资源节点；Focus Mode 提供专注计时和前台窗口检测的扩展接口；复盘功能根据历史记录和节点状态生成项目进展总结。
8. 保存与加载。任务图、桌宠状态、工作区状态和历史记录都可保存为 JSON。程序支持默认图文件、样例图和用户保存文件，便于继续编辑和演示。

2 项目各模块与类设计细节

项目整体采用分层设计，核心目录为 `src/petflow/`。主要分为 UI 层、Application 层、Domain 层、Repository 层、Service 层、Agent 层和 System 集成层。这样的划分使界面交互、业务规则、数据存储和外部能力互相隔离，便于多人协作和单元测试。

2.1 Domain 层：核心数据模型与规则

Domain 层位于 `src/petflow/domain/`，不依赖 Tkinter、文件系统或网络请求，是项目中最稳定的核心部分。

`entities.py` 定义主要实体类：

- **Node**：表示图中的节点。字段包括 `id`、`type`、`title`、`description`、`status`、`priority`、`estimated_minutes`、`actual_minutes`、坐标、标签、附件、重复规则、资源路径、检查清单等。`to_dict()` 和 `from_dict()` 负责 JSON 序列化与反序列化。
- **Edge**：表示节点之间的关系。包含 `source`、`target`、`type`、`label`、`weight` 等字段。
- **ChecklistItem**：表示节点中的检查项。
- **ProjectMetadata**：保存项目名称、描述、schema 版本和更新时间等元数据。
- **PetState**：保存桌宠当前节点、位置、状态、心情和气泡文本。
- **WorkspaceState**：保存当前选择、缩放、平移、主题和专注模式等界面状态。

`enums.py` 用枚举集中管理字符串常量，例如 `NodeType`、`NodeStatus`、`EdgeType`、`RepeatType`、`ResourceType`、`PetStateType` 和 `EventType`。这样可以避免在代码中散落 "todo"、"dependency" 等硬编码字符串。

`graph.py` 中的 `GraphModel` 是任务图模型，维护所有节点、边、桌宠状态、工作区状态和历史记录。它提供 `add_node()`、`update_node()`、`remove_node()`、`add_edge()`、`update_edge()`、`remove_edge()`、`predecessors()`、`successors()`

等方法。图模型会检查节点和边 ID 是否重复、边端点是否存在、删除节点时是否同步删除关联边，以及依赖边是否会造成环。

`rules.py` 和 `routine.py` 封装领域规则。依赖关系的成环检测放在领域层，`Routine` 的下次到期时间计算也放在领域层，避免 UI 回调中混入业务规则。

2.2 Application 层：用例编排与事件分发

Application 层位于 `src/petflow/app/`，负责把 UI 操作转换为对领域模型的安全修改。

`GraphService` 是最重要的应用服务，也是 UI 和 Agent 修改任务图的统一入口。它封装了创建节点、创建资源节点、添加附件、更新节点详情、更新节点状态、移动节点、删除节点、创建边、更新边、删除边、设置当前节点等操作。每次修改后，`GraphService` 会发布事件，便于桌宠、界面刷新和其他服务响应。

`GraphService.update_node_status()` 对状态流转做了集中处理。当节点被标记为 `done` 时，会写入完成时间；如果是 `Routine` 节点，还会更新 `last_completed_at`、`next_due_at` 和连续完成次数 `streak`，并记录历史事件。

`EventBus` 是同步事件总线。它通过发布和订阅 `DomainEvent` 解耦模块，例如节点更新后桌宠可以自动响应，而不需要 UI 直接调用桌宠逻辑。

`AppContext` 是应用上下文，用于集中创建和保存 `GraphModel`、`GraphService`、`StorageService`、`RecommendationEngine`、`AgendaService`、`ReviewService`、`PetService` 等对象。主窗口只需要持有 `AppContext`，就可以访问各项服务。

2.3 Repository 与存储层

Repository 层位于 `src/petflow/repositories/`，Service 层中的 `StorageService` 对其进行封装。

`GraphRepository` 是抽象接口，定义 `load()` 和 `save()` 方法。`JsonGraphRepository` 是当前实现，负责从 JSON 文件读取任务图，或把 `GraphModel.to_dict()` 的结果写入本地文件。默认图文件路径为 `data/graph.json`。

这种设计避免 UI 直接读写文件。以后如果要支持“另存为”“导入”“导出”或其他存储格式，只需要替换或扩展 `Repository` 实现。

2.4 Service 层：业务能力服务

Service 层位于 `src/petflow/services/`，用于放置不直接属于实体模型、但又需要被多个界面入口复用的业务逻辑。

`RecommendationEngine` 负责推荐下一步任务。它先排除已完成和 `blocked` 节点，再检查依赖前置是否完成。评分时会考虑优先级、`doing` 状态、`paused` 状态、`Routine` 到期、奖励节点和当前焦点节点，最后返回分数最高的候选节点，并能生成推荐理由。

AgendaService 负责生成未来若干天的议程。它读取节点的 `next_due_at` 和重复规则，把任务按日期归组，并按优先级和标题排序。资源节点和已完成节点不会进入议程。

RoutineService 负责 **Routine** 到期判断和批量刷新。**ReviewService** 根据图历史、节点状态和 **Routine** 信息生成复盘数据。**ResourceService** 整理资源节点打开、复制和预览所需的信息。**GraphLayoutService** 负责节点自动排列和 **Agent** 生成子图后的布局调整。

PetService 负责桌宠状态响应。它订阅节点更新事件，当节点进入 `doing` 状态时移动到该节点，当节点完成时调用推荐引擎寻找下一步，并把桌宠设置为 `happy` 或 `idle` 状态。**PetService** 只更新 **PetState** 并发布 `PET_MOVED` 事件，具体绘制交给 **UI** 层。

MascotService 负责加载桌宠配置。它从 `assets/mascots/` 和 `data/mascots/` 中读取 `mascot.json`，检查图片资源是否存在，并生成 **MascotConfig**。

2.5 UI 层：Tkinter 界面与交互

UI 层位于 `src/petflow/ui/`。它的职责是显示数据、收集用户操作，并把操作交给应用服务处理。

MainWindow 是主窗口类，负责创建 **Tk** 根窗口、顶部工具栏、推荐横幅、左侧议程面板、中心图画布、右侧编辑/伴侣面板、主题切换、**Focus Mode**、保存加载、**Agent** 对话框入口等。它还负责在文件加载后重建 **AppContext**，保证界面和服务使用同一份图模型。

GraphCanvas 继承自 `tk.Canvas`，负责图的可视化绘制和交互。它维护 `canvas item` 到 `node/edge ID` 的映射，支持节点绘制、边绘制、节点拖拽、选择、高亮、缩放、平移、边创建模式、右键菜单、完成动效和图展开动画。**Canvas** 不直接绕过服务修改业务数据，而是通过 **GraphService** 完成节点移动、删改和连边。

InspectorPanel 是右侧内联编辑器。选中节点或边后，用户可以在这里修改标题、类型、状态、优先级、预计耗时、实际耗时、描述、标签、资源路径、重复设置、到期日期以及边类型和边标签。它同样通过 **GraphService** 保存修改，并保留高级对话框作为补充入口。

AgendaPanel 展示未来 7 天计划和保存图文件中的任务。**PetAssistantPanel** 提供桌宠伴侣聊天和 **Agent** 辅助入口。**AgentDialog** 负责展示 **Agent** 生成的 `proposal` 预览，并在用户确认后调用 **AgentExecutor** 应用。**SettingsDialog** 用于配置 **API Key**、**Base URL**、模型和 `mock` 模式。**IconButton**、`components.py`、`theme.py` 和 `fonts.py` 提供可复用的界面组件、主题色和字体设置。

2.6 Agent 层：自然语言规划与校验执行

Agent 层位于 `src/petflow/agent/`，分为 Prompt、客户端、proposal 校验和执行四部分。

`PromptBuilder` 负责根据当前图状态和用户输入构造提示词。`AgentClient` 负责访问 OpenAI 兼容 API，支持 chat completions 和 responses 两种接口，也支持无 API Key 时回退 mock 模式。它会要求模型返回 JSON，并对响应进行 JSON 解析。

`AgentProposalValidator` 负责校验 Agent 输出。它限制最大节点数，校验节点标题、类型、状态、优先级、耗时、重复规则、资源类型、坐标、边端点等字段，并兼容部分常见别名，例如 `completed` 可归一化为 `done`。

`AgentExecutor` 负责把校验后的 proposal 应用到图中。它不会直接修改 `GraphModel`，而是调用 `GraphService.create_node()`、`create_edge()`、`update_node_detail()` 和 `delete_node()` 等方法，因此仍然会经过 ID 生成、字段校验、成环检测和事件发布。对于节点拆分场景，Executor 会自动在父节点和新节点之间添加推荐边。

2.7 System 集成层

System 层位于 `src/petflow/system/`，用于封装和操作系统相关的能力。

`ClipboardWatcher` 提供一次性剪贴板捕获。它读取剪贴板文本，判断内容是否为 URL，并生成 `ClipboardCapture`，后续由 `GraphService.create_resource_node()` 转成资源节点。

`FocusMonitor` 提供前台窗口标题检测。目前该能力只在 Windows 且安装 `pywin32` 时可用；在 macOS 或未安装依赖时会安全返回不可用，不影响主程序启动。

3 小组成员分工情况

成员	主要负责内容
王显然	基础功能完善、agent 功能开发
王雨川	基础功能设计、agent 功能完善
万亚骅	pet 形象设计、视频剪辑

4 项目总结与反思

本项目的主要收获是把“待办事项管理”设计成了一个可视化任务图系统。相比简单列表，图结构更适合表达前置依赖、循环 Routine、任务拆分和推荐路径。项目中还加入了桌宠、七日议程、Agent 辅助规划、剪贴板资源捕获和复盘等功能，使程序既有可演示性，也有一定实用价值。

从工程实现上看，分层设计带来了明显好处。`GraphModel` 只关心图结构和领域规则，`GraphService` 统一处理修改入口，UI 层只负责交互和展示，Agent 层也不能绕过服务直接改数据。这样做虽然前期代码量更多，但后期增加推荐、桌宠、日程和 Agent 时，不需要反复修改底层结构。

项目也暴露出一些不足。第一，UI 层功能较多，`MainWindow` 和 `GraphCanvas` 的代码仍然偏大，后续可以继续拆分为更小的控制器或组件。第二，Agent 生成结果虽然已经做了校验，但对复杂自然语言目标的布局 and 任务粒度控制还有改进空间。第三，Focus Mode 的系统检测能力目前主要面向 Windows，在跨平台体验上还不完整。第四，桌宠交互已经能响应任务状态，但还可以加入更丰富的动画和更细致的反馈策略。

后续如果继续完善，可以优先从三个方向推进：一是继续优化 UI 结构和视觉细节，提高长期使用时的效率；二是增强 Agent proposal 的预览、撤销和冲突处理能力；三是完善跨平台系统能力，让剪贴板、专注检测、文件资源和复盘形成更完整的工作流闭环。